

Organização e Arquitetura de Computadores

Estrutura da CPU

Prof. Msc. Lucas Reis

lucas.oliveira@puc-campinas.edu.br



Nas aulas anteriores...

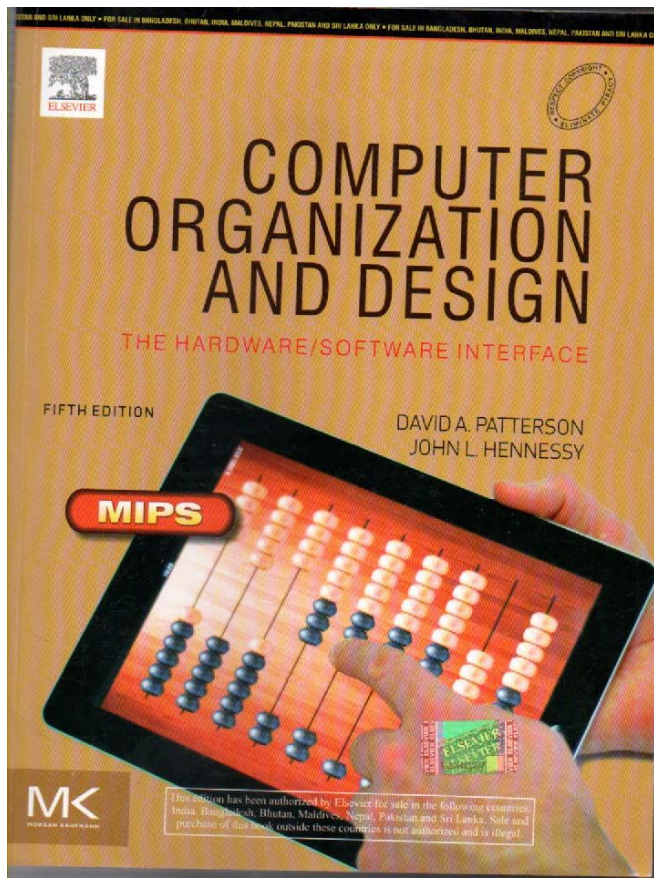
```
leaf:  
    add    $t0, $a0, $a1  
    add    $t1, $a2, $a3  
    sub    $v0, $t0, $t1  
    jr     $ra
```

- Entendemos como assembly funciona em MIPS
- Nessa aula, veremos a estrutura de um processador MIPS32 e como essas instruções são interpretadas em um nível mais baixo

+ Recomendação de Leitura

IMPORTANTE:

Essa aula exige uma quantidade de informações que é impossível carregar nos slides. Mais do que nunca, recomenda-se o livro



Organização e Design de Computadores

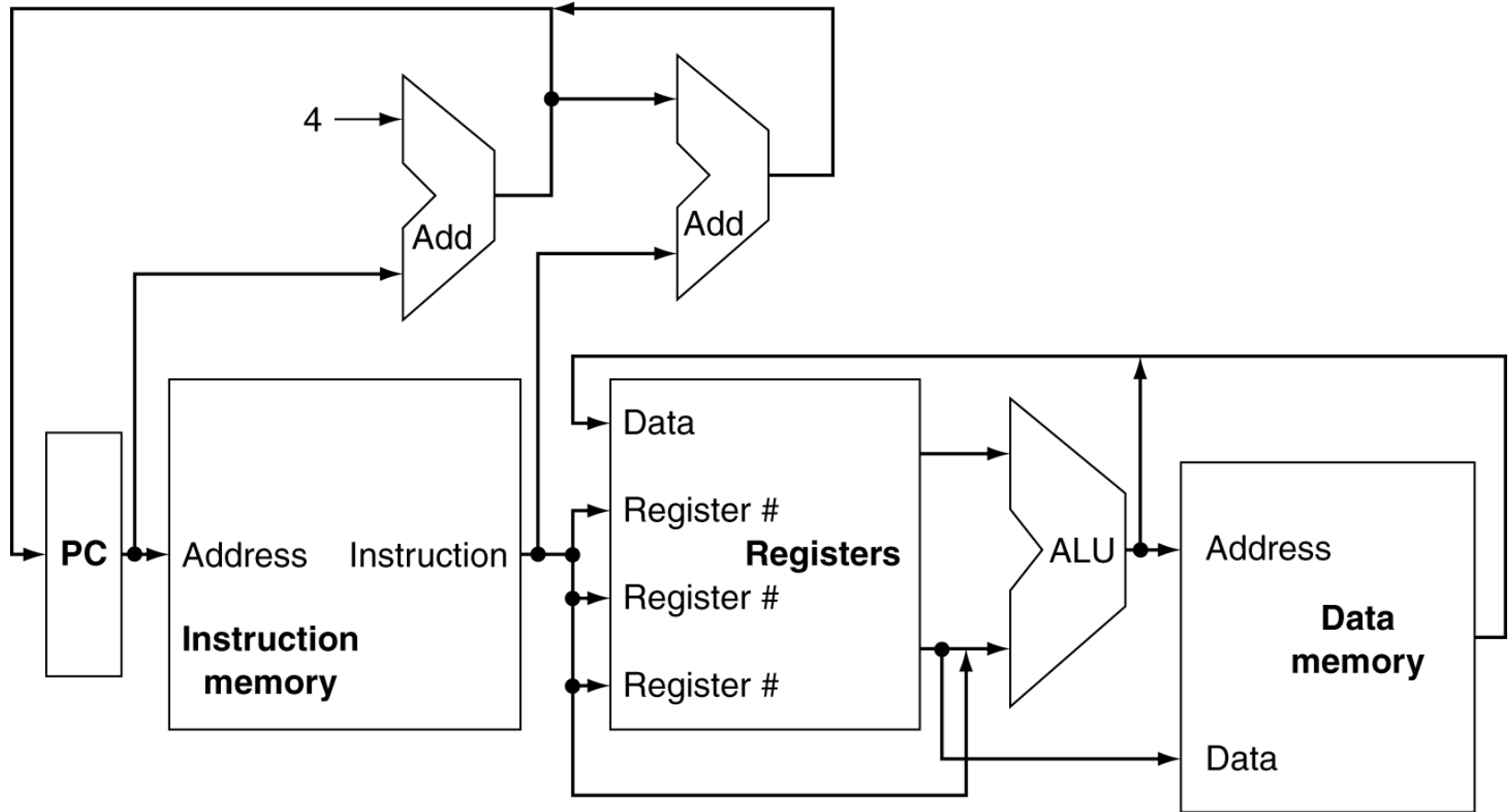
Hennesy & Patterson

Nesta aula: **Capítulo 4**

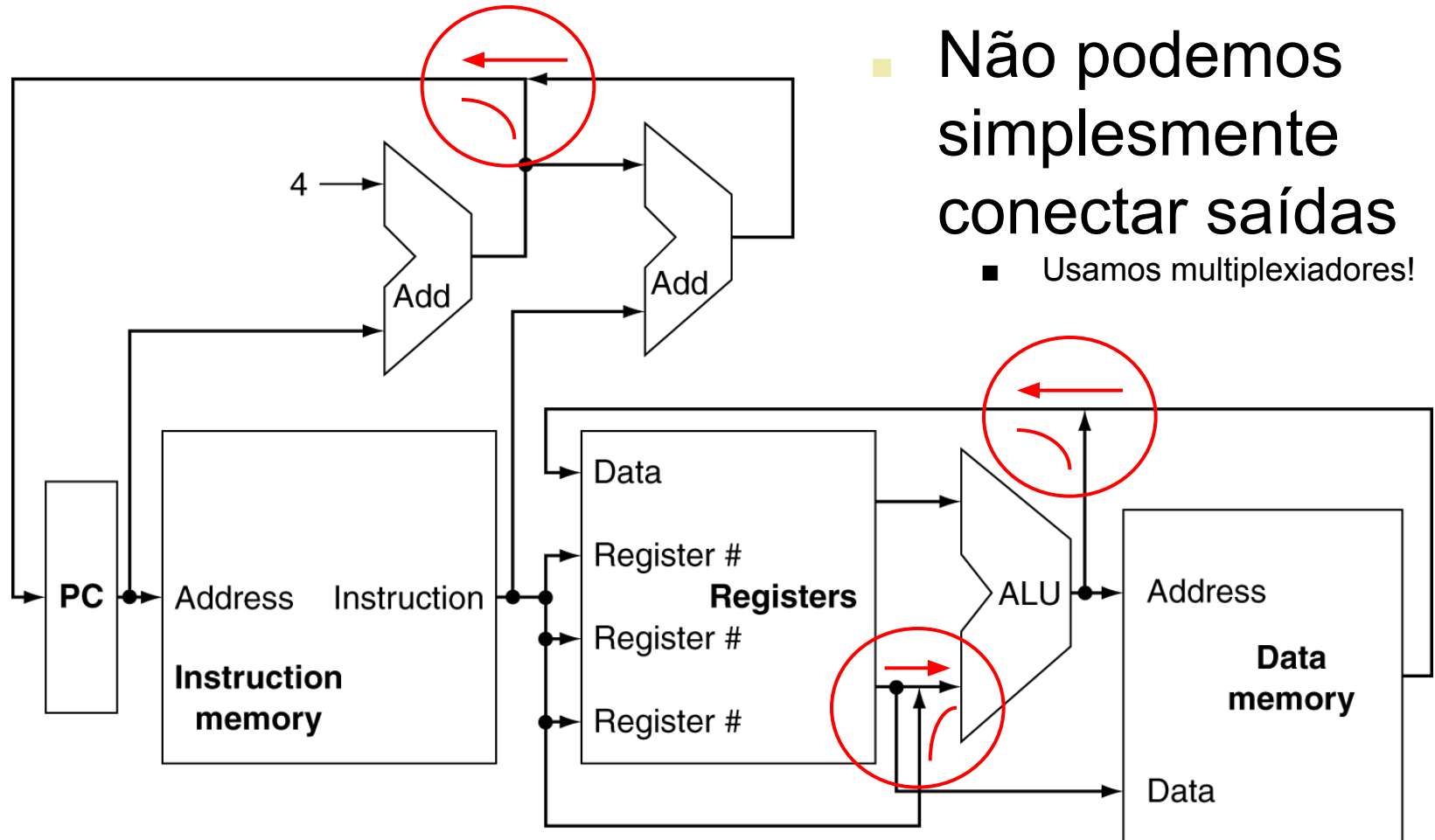
Execução de Instruções

- PC → busca instrução na memória de instruções
- Registradores → busca do banco de registradores
- Dependendo da instrução
 - Usar ALU para calcular
 - Resultado aritmético
 - Endereço de memória para load/store
 - Endereço de destino de um desvio
 - Acessar memória para load/store
 - $PC \leftarrow$ endereço do desvio ou $PC + 4$
 - **Por que +4?**

Visão geral de uma CPU

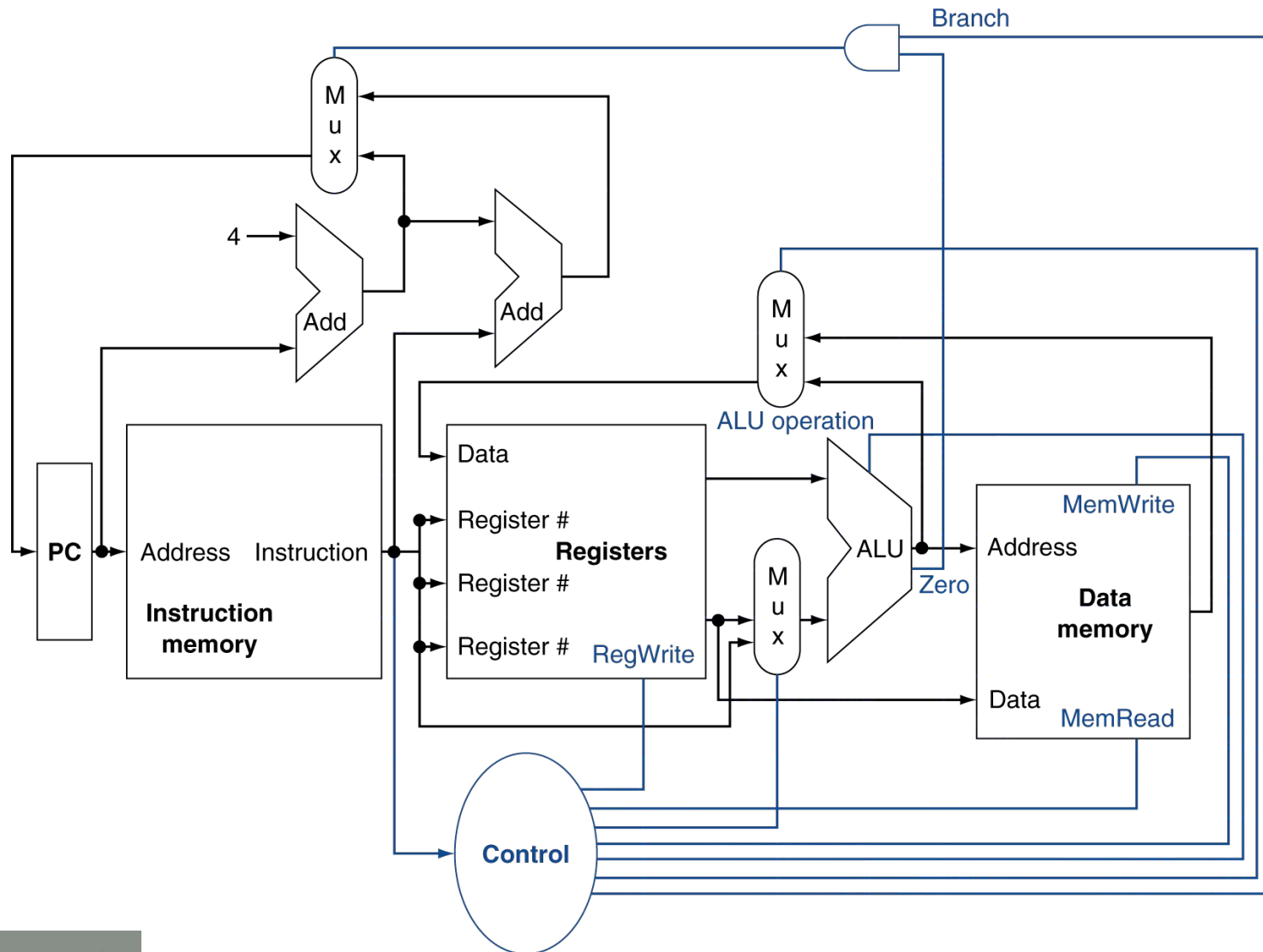


Multiplexadores



- Não podemos simplesmente conectar saídas
 - Usamos multiplexiadores!

Adicionando controle

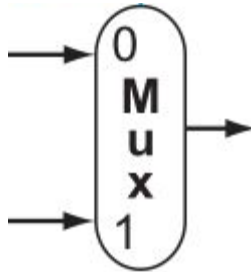


Design lógico básico

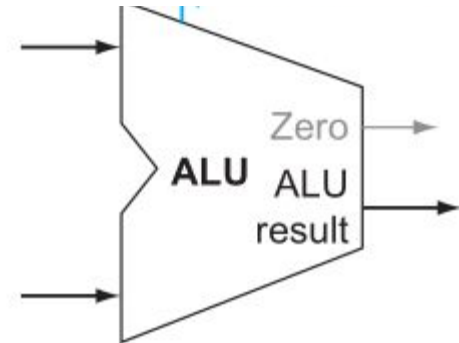
- Informação codificada em binário
 - Baixa tensão = 0, Alta tensão = 1
 - Um fio por bit
 - Dados multi-bit/byte codificados em barramentos multifios
- Elementos combinacionais
 - Operam em dados
 - Saída é função das entradas
- Elementos de estado (sequenciais)
 - Armazenam informações

Elementos Combinacionais

- Multiplexador



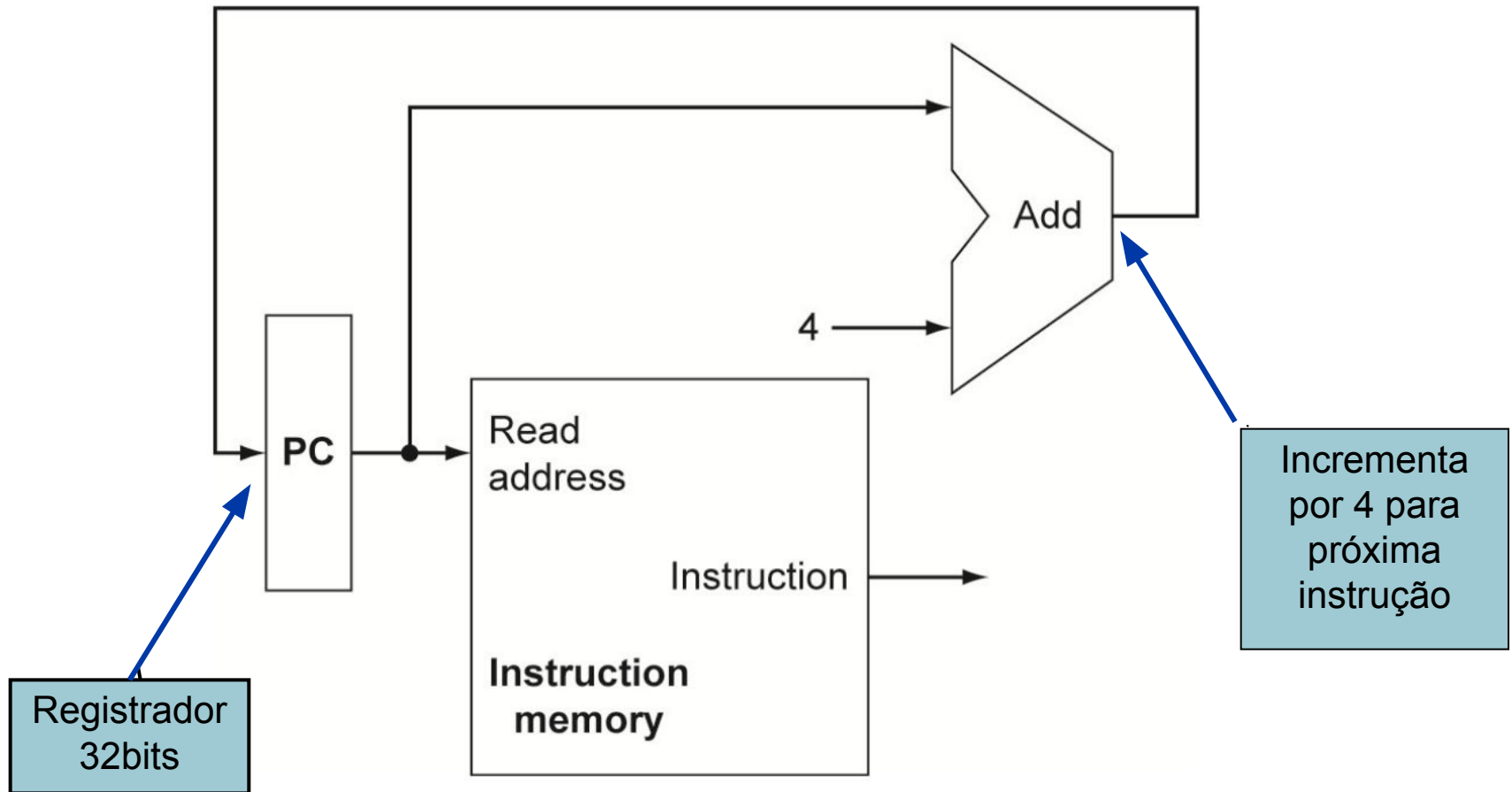
- ALU



Construindo o Datapath

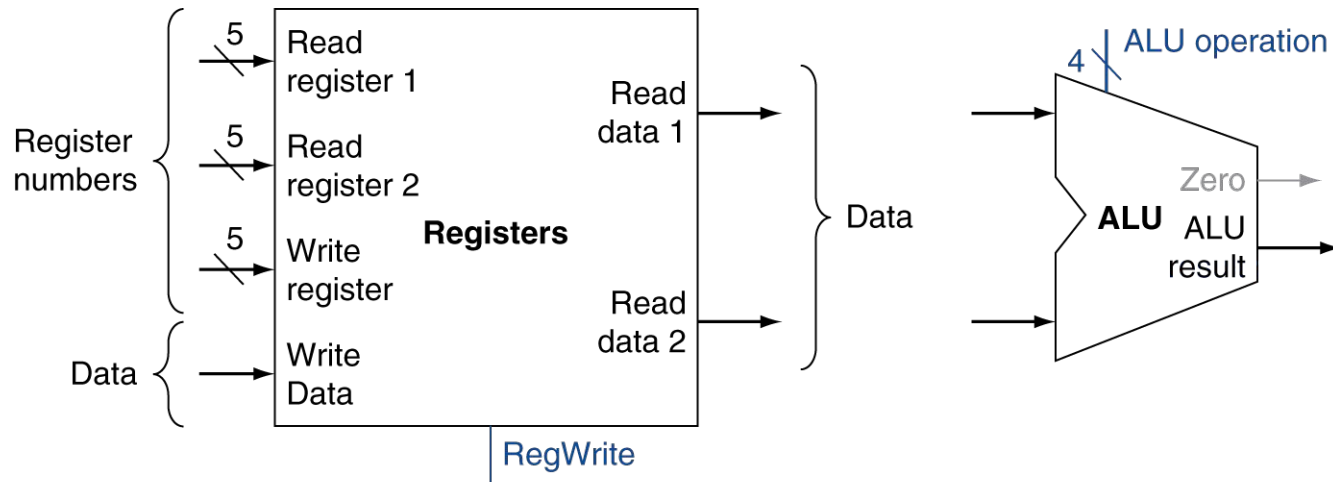
- Datapath
 - Elementos que processam dados e endereços na CPU
 - Registradores, ALUs, multiplexadores, memórias, ...
- Vamos construir, incrementalmente, o datapath de uma versão simplificada do MIPS32
 - Instruções suportadas: add, sub, lw, sw, beq

Busca/Fetch de Instrução



Instruções em formato R

- Instruções lógicas/aritméticas
 - add, sub, and, or, ...
- Lê dois registradores
- Realiza operação lógica/aritmética
- Guarda resultado em registrador

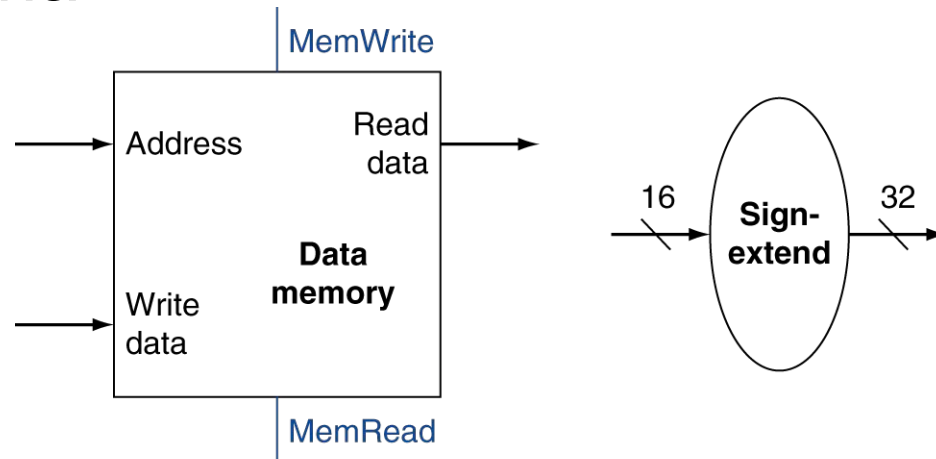


a. Registers

b. ALU

Instruções Load/Store

- Lê operandos dos registradores
- Calcula endereço usando offset de 16 bits
 - Usa ALU para isso, mas estende o tamanho do offset
- Load: Lê da memória, atualiza registrador
- Store: escreve de um registrador para a memória



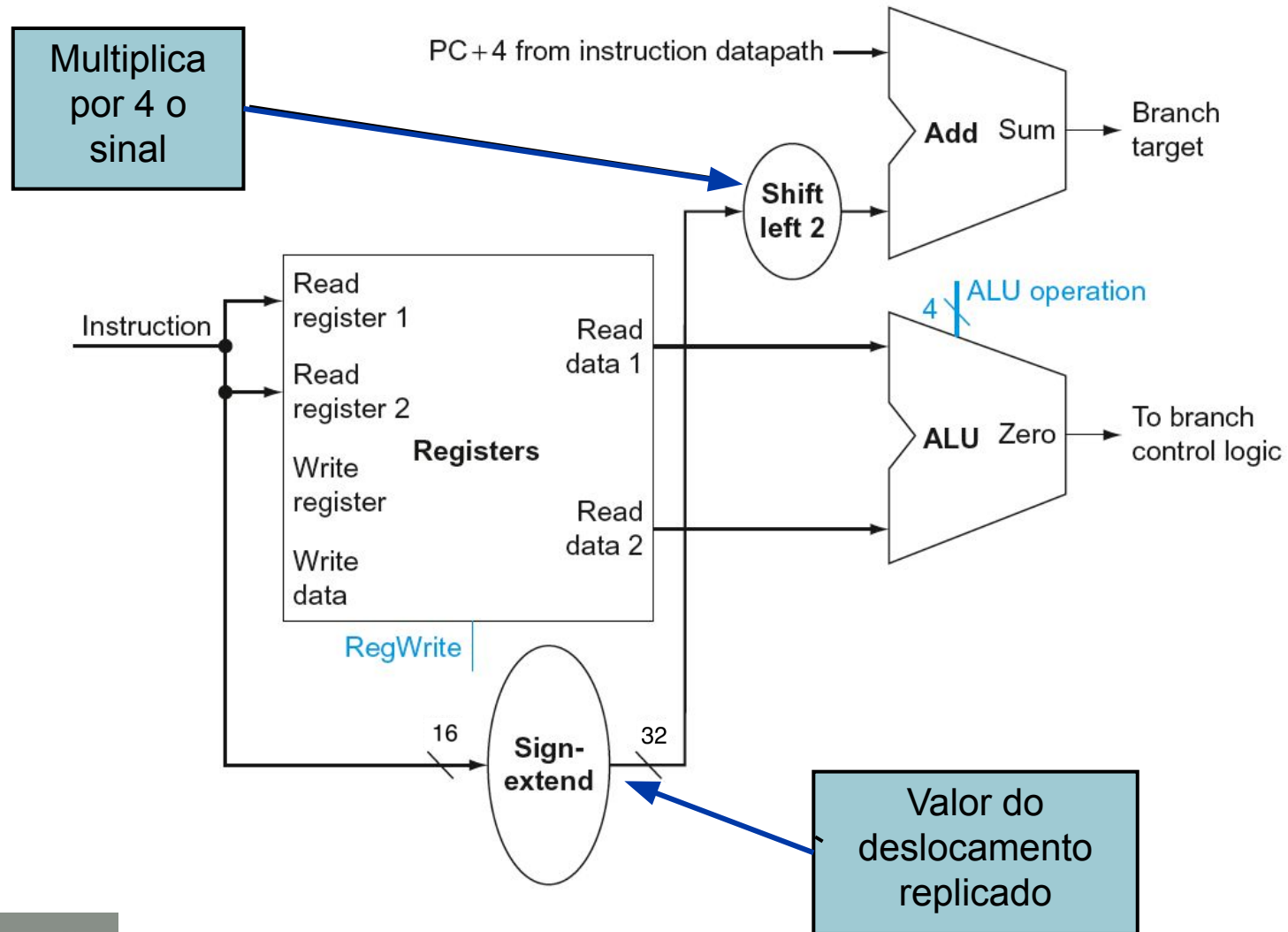
a. Data memory unit

b. Sign extension unit

Instruções de Salto

- Lê operandos dos registradores
- Compara operandos
 - Usa ALU, subtrai e checa se a saída é Zero
- Calcula endereço do desvio
 - Estende o endereço do desvio
 - Shift left 2 vezes (multiplica por 4)
 - **Por que?**
 - Soma a $PC + 4$
 - Já foi calculado pelo fetch anteriormente

Instruções de Salto



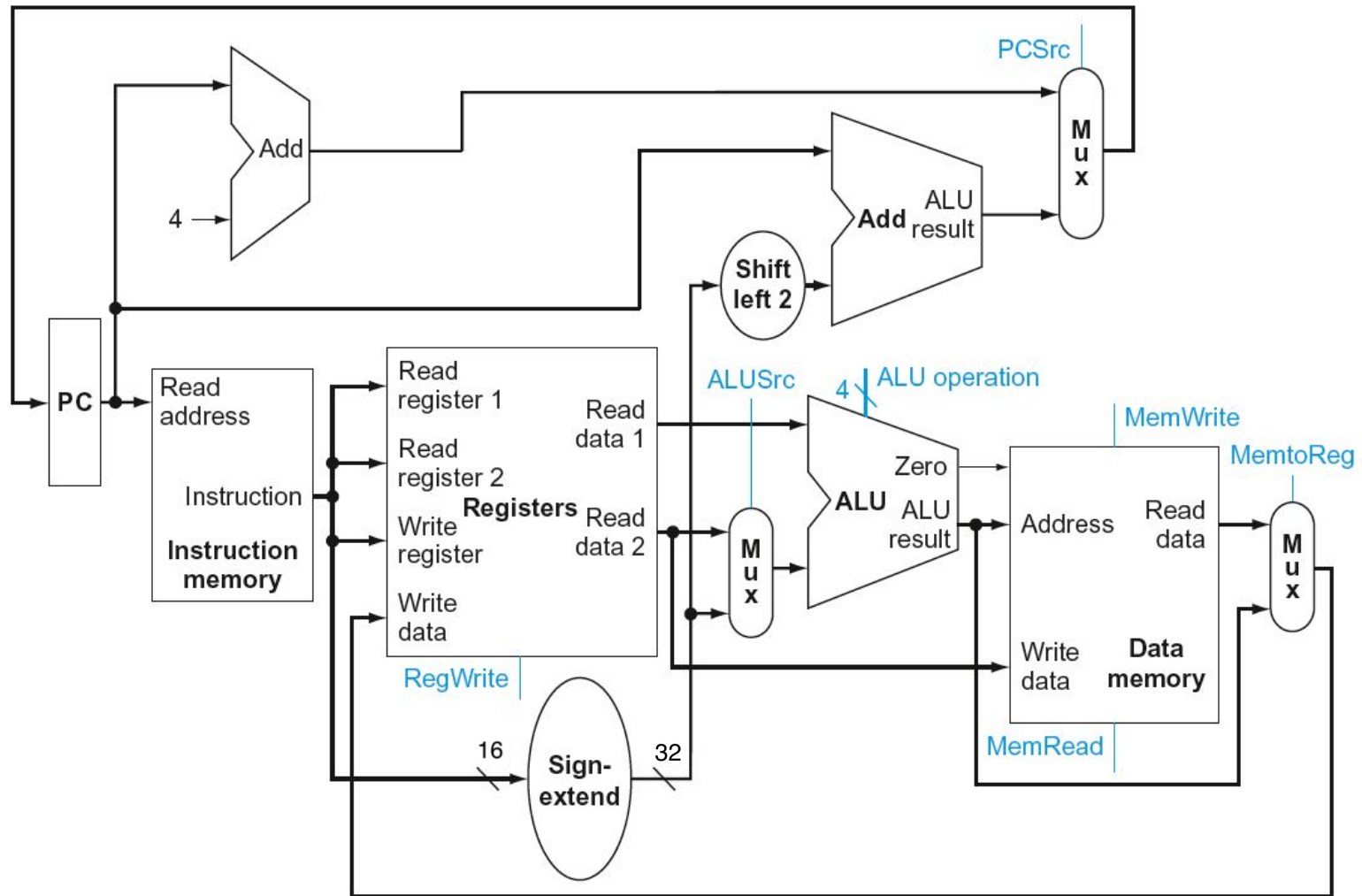
Combinando os elementos

- Datapath faz uma instrução por ciclo
 - Cada elemento só pode fazer uma função por vez
 - Portanto, precisamos separar a memória de instruções e a memória de dados
 - Na realidade, sabemos que não é assim...
- Usar multiplexadores onde dados de fontes diferentes são usados para um mesmo elemento

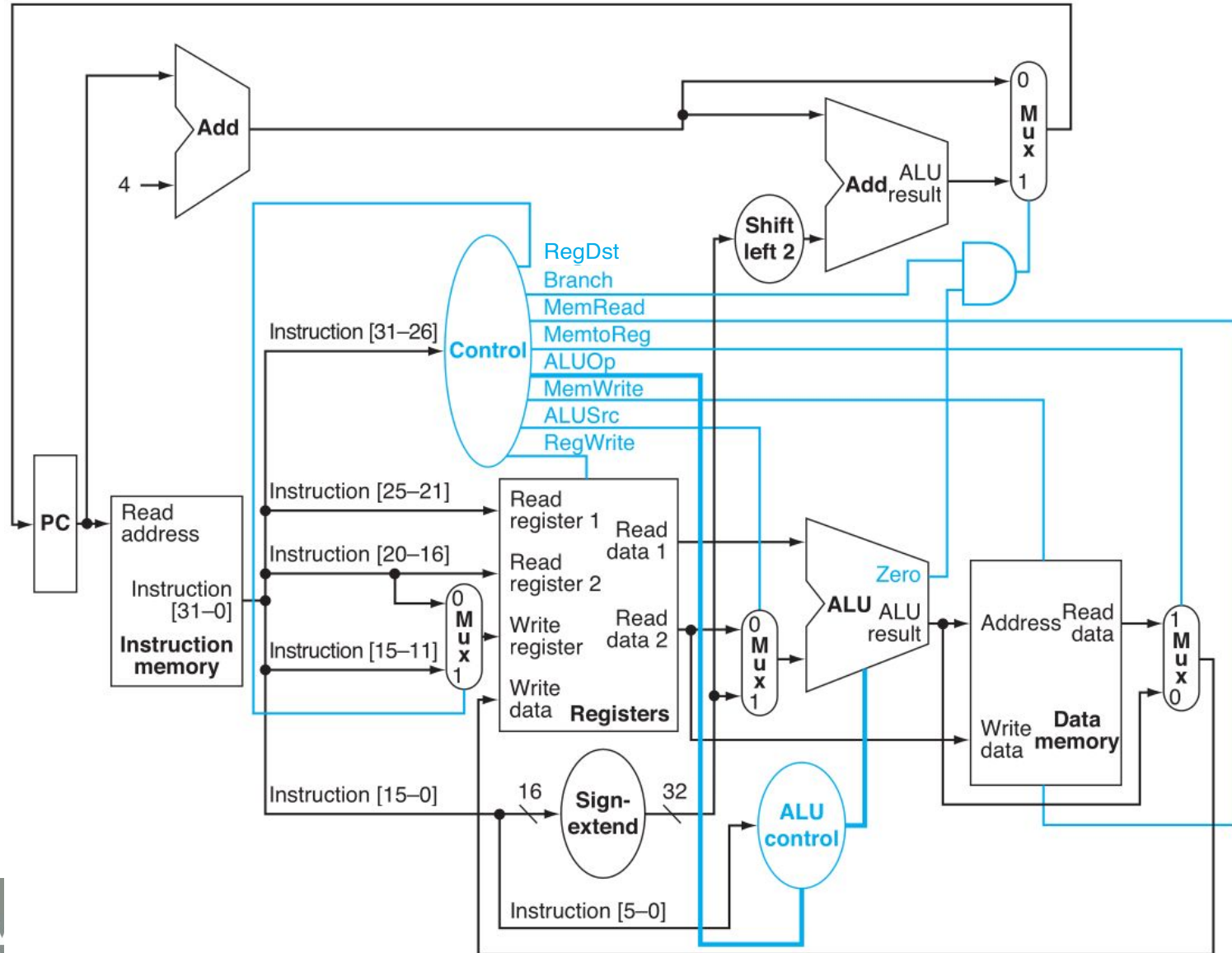
MK



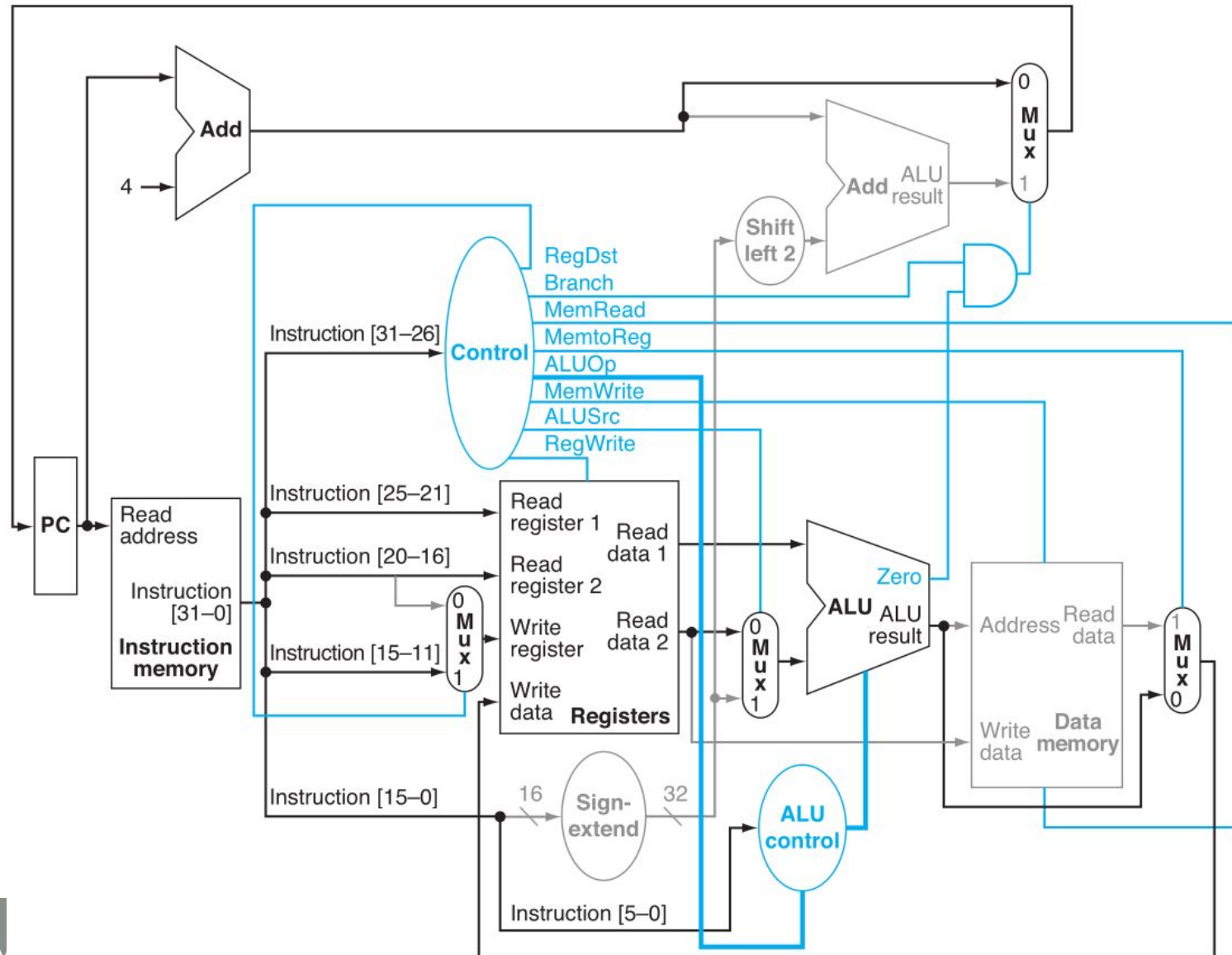
Datapath Completo



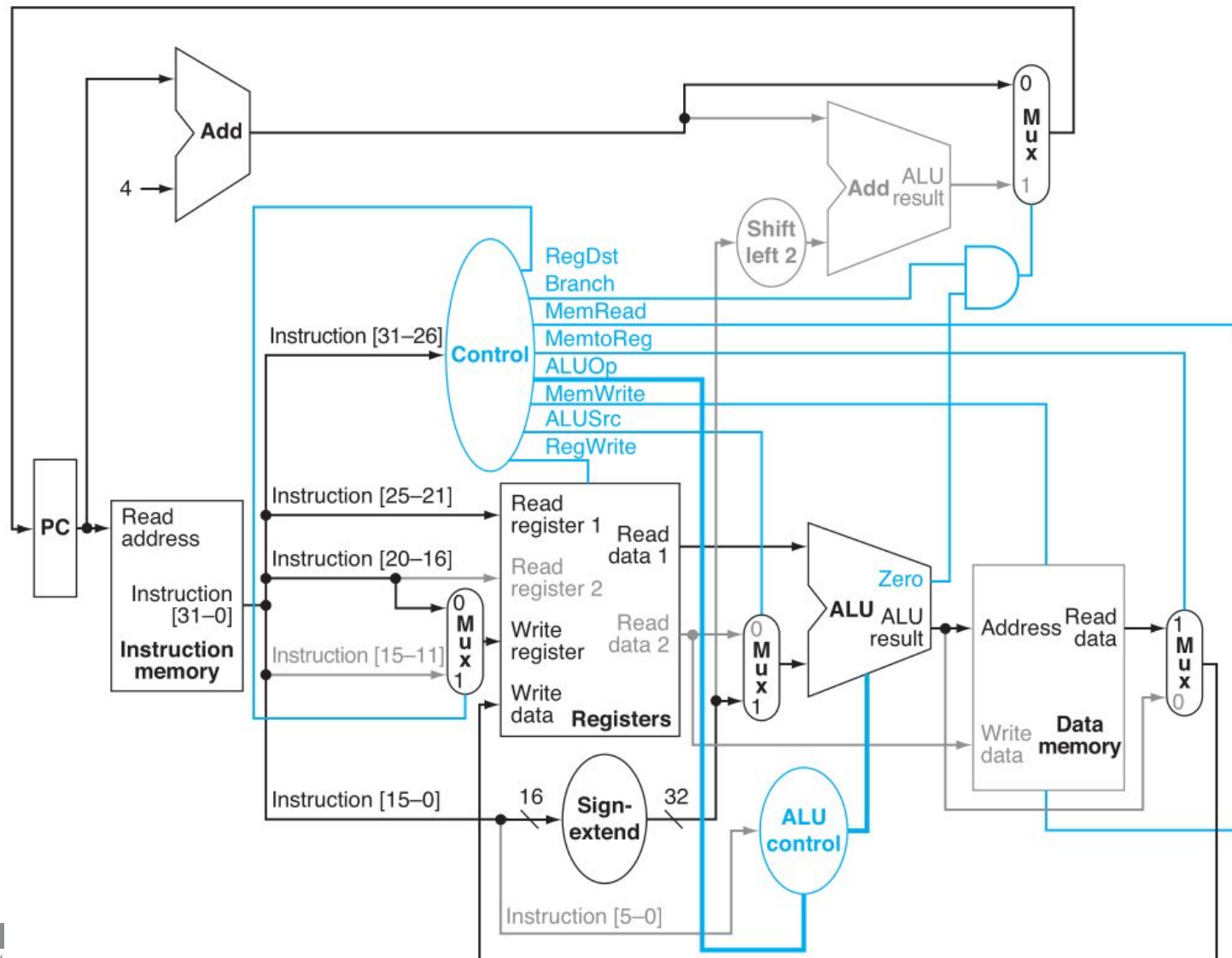
Datapath com Controle



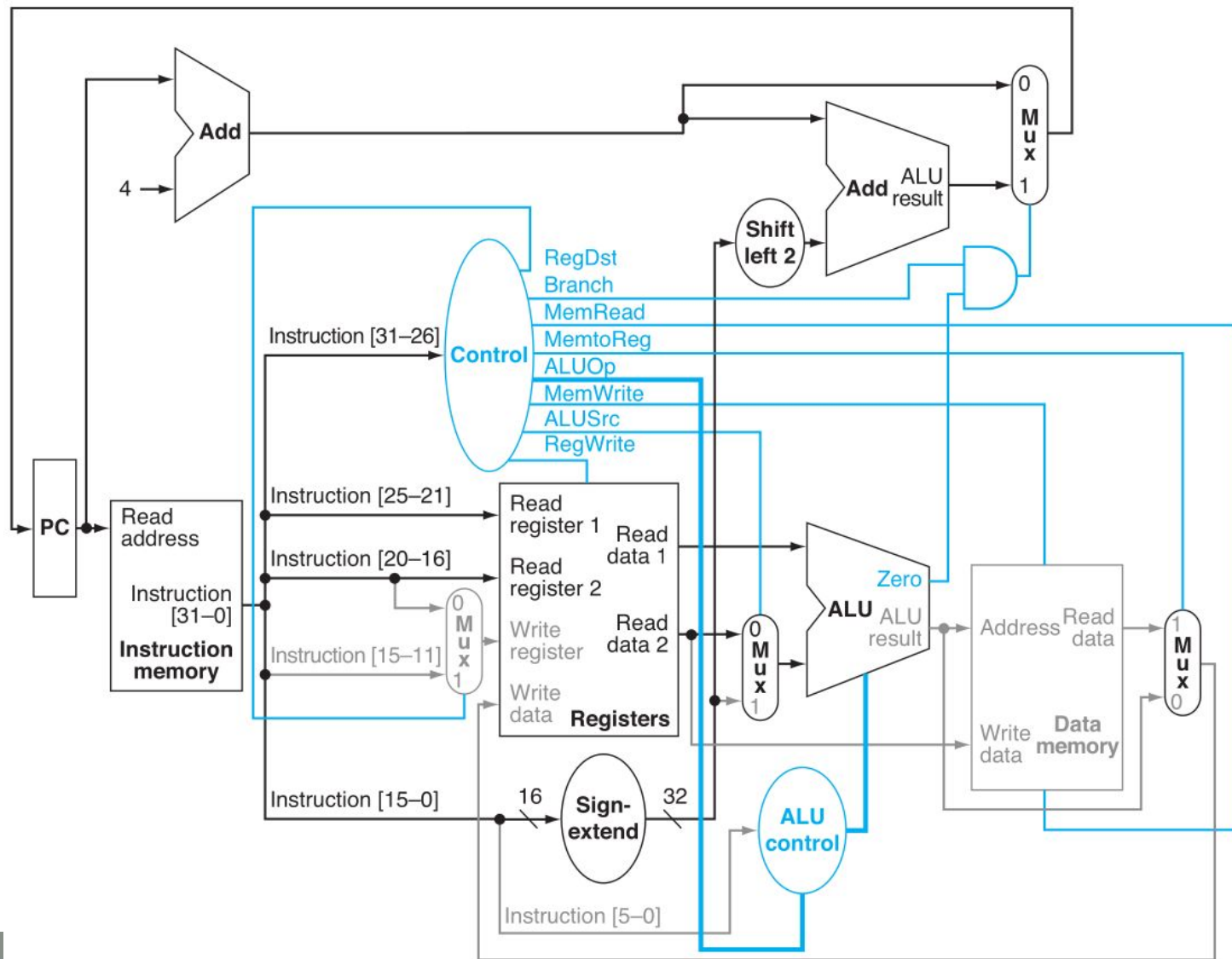
Instrução tipo R



Instrução de Load



Instrução beq

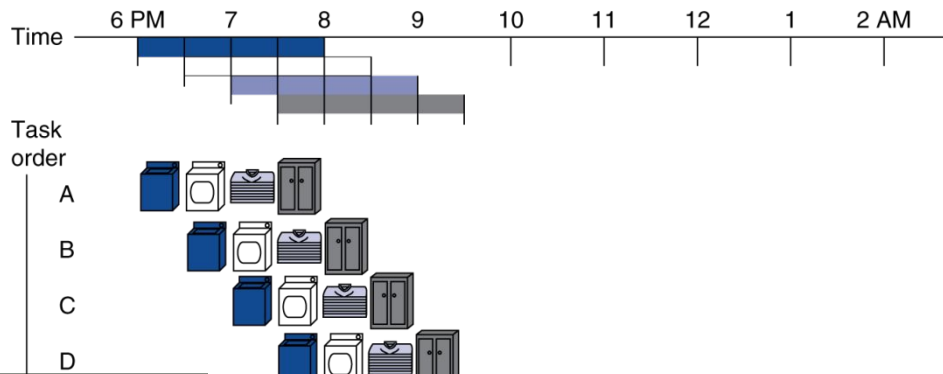
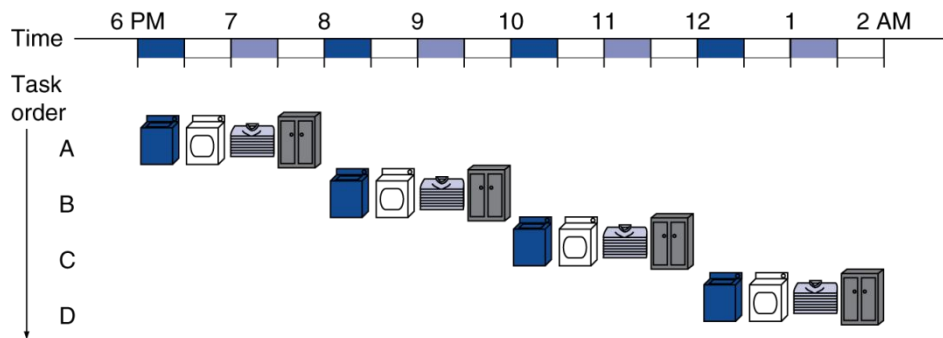


Problemas de desempenho

- **Caminho crítico** determina o período do clock
 - Neste datapath: instrução de load
 - Memória de instruções → banco de registradores → ALU → memória de dados → banco de registradores
- Inviável alterar o período para instruções diferentes
- Como melhorar?
 - **Pipelines!**

Analogia para Pipeline

- Lavar roupa em pipeline: execução sobreposta
 - Paralelismo melhora desempenho



- Quatro levadas:

- $\text{Speedup} = 8 / 3.5 = 2.3$

- Sem parar:

- $\text{Speedup} = 2n / 0.5n + 1.5 \approx 4$
= número de estágios

Pipeline MIPS

- Cinco estágios, um passo por estágio
 1. IF: Instruction Fetch - busca da instrução na memória
 2. ID: Instruction decode & leitura de registradores
 3. EX: Executa operação ou calcula endereço
 4. MEM: Acesso ao operando de memória
 5. WB: Write Back - Escreve resultado de volta

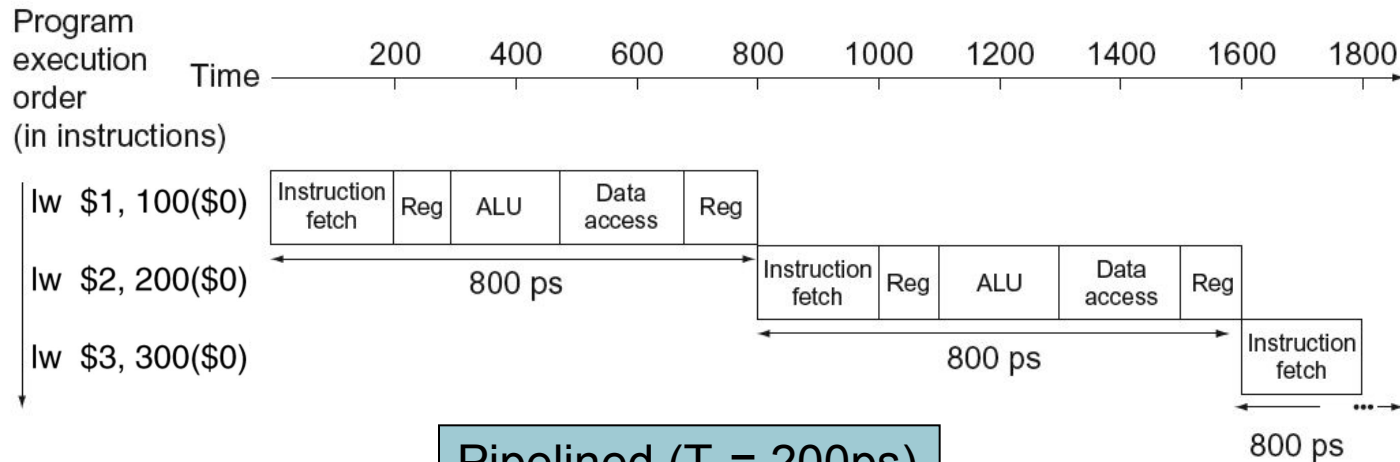
Desempenho do Pipeline

- Assumindo tempo de acesso como:
 - 100ps para registradores
 - 200ps para memória e ALU

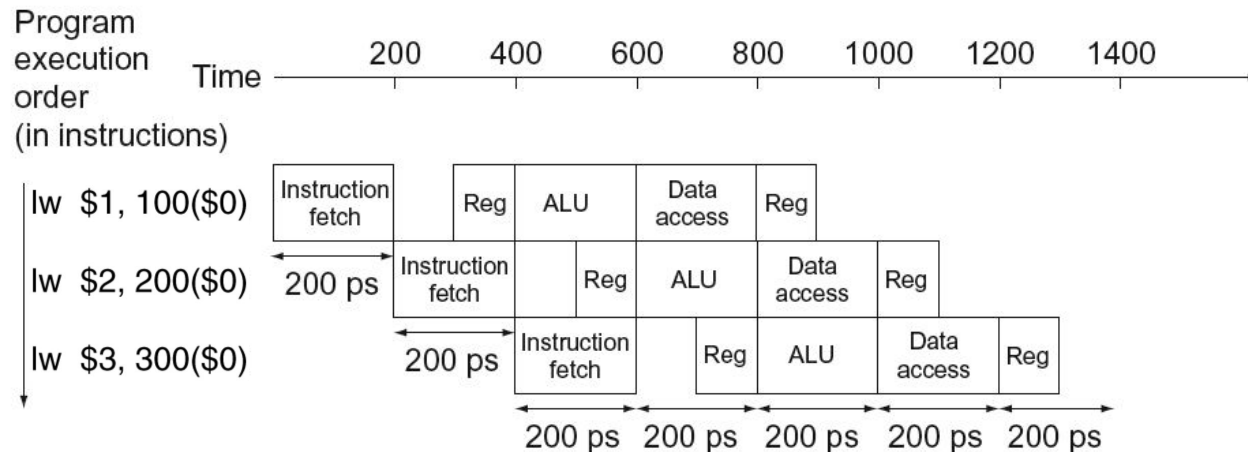
Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
ALU	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Desempenho do pipeline

Single-cycle ($T_c = 800\text{ps}$)



Pipelined ($T_c = 200\text{ps}$)



Hazards

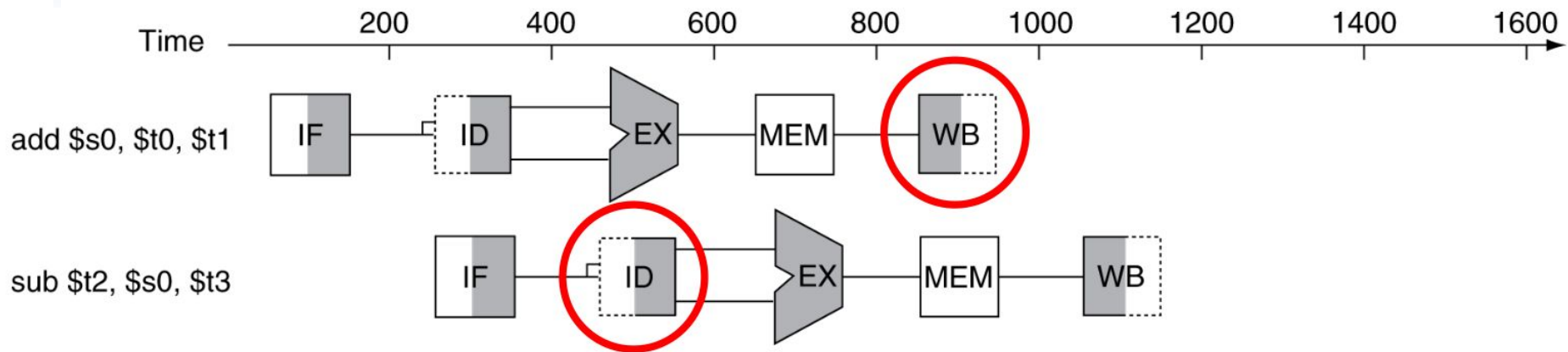
- Situações que previnem começar a instrução do próximo ciclo
- Hazard Estrutural
 - Um elemento do processador está ocupado
- Hazard de dados
 - O dado necessário ainda não está pronto
- Hazard de Controle
 - Uma decisão de controle depende da instrução anterior

Hazard Estrutural

- Conflito no uso de um recurso
 - Load/store precisam acessar dados
 - Fetch de instruções teria que parar (*stall*) por aquele ciclo
 - Causa “bolha” no pipeline
- Dataphs em pipeline exigem memória separada para instrução e dados
 - Ou cache separada para instrução e dados

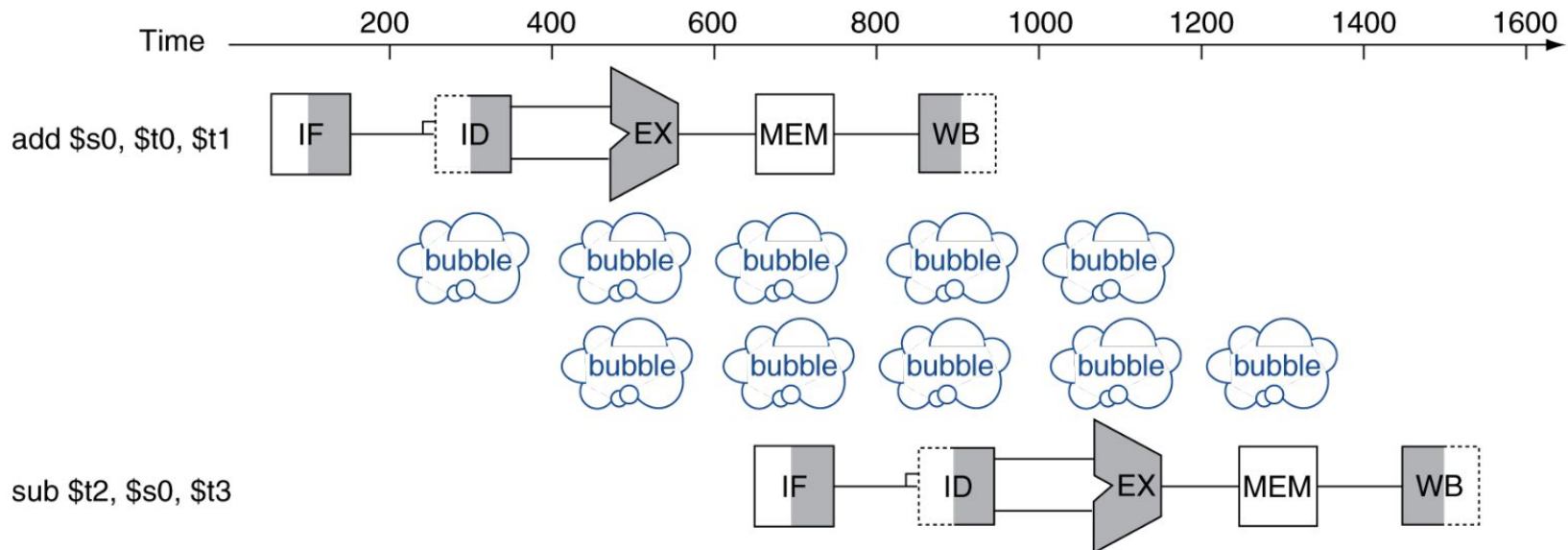
Hazard de Dados

- Uma instrução depende da completude do acesso de dado de uma instrução anterior
 - add **\$s0**, \$t0, \$t1
 - sub \$t2, **\$s0**, \$t3



Hazard de Dados

- Uma instrução depende da completude do acesso de dado de uma instrução anterior
 - add **\$s0**, \$t0, \$t1
 - sub \$t2, **\$s0**, \$t3



Paralelismo a Nível de Instrução (ILP)

- Pipeline: executar várias instruções em paralelo
- Para aumentar o ILP
 - Pipeline mais “profundo”
 - menos trabalho por estágio $\Rightarrow$ ciclo de clock menor
 - Despacho múltiplo
 - Replicar estágios do pipeline $\Rightarrow$ vários pipelines
 - Começar várias instruções por clock
 - $CPI < 1$, então usamos Instruções por Ciclo (IPC)
 - Dependências e hazards reduzem o número de instruções em paralelo
 - Execução especulativa
 - Começar a executar uma instrução mesmo sem ter os dados, assumindo valores
 - Cancelar e voltar se estiver errado

Para praticar!

```
LOOP: slt  $t2, $0, $t1  
      beq  $t2, $0, DONE  
      subi $t1, $t1, 1  
      addi $s2, $s2, 2  
      j   LOOP
```

DONE:

Suponha que o registrador **\$t1** seja inicializado com o valor **10**. Mostre no caminho de dados a execução da instrução **beq**, preenchendo os valores dos sinais de dados e controle.

